Оглавление

Алгоритмические языки 2

Язык C/C++ 4

Простейшие программы на языке C++ 9

Преобразование типов 16

Статическая типизация и неявное преобразования типов в C++ 20

Указатели 29

Структуры в C/C++ 31

Символы и строки в C++ 35

Низкий уровень 41

Область видимости (Scope) переменных 43

Работа с файлами в C++ 45

Форматированный ввод/вывод (C-стиль) 47

Работа с файлами в C (stdio.h) 48

Алгоритмические языки

Алгоритмический язык — это формальный язык, используемый для записи, реализации и изучения алгоритмов.

Язык программирования — это формальная знаковая система для записи компьютерных программ.

Программа — это совокупность данных и команд, предназначенных для реализации определенного алгоритма.

Программирование — это научная и практическая деятельность по созданию программ.

Язык программирования определяет набор лексических, синтаксических и семантических правил, задающих внешний вид программы и действия, которые выполнит компьютер под её управлением.

Синтаксис устанавливает правила построения всех элементов языка и описывает структуру программы как набор символов.

Семантика определяет смысл и правила использования каждой синтаксической конструкции.

Программа, непосредственно исполняемая процессором компьютера, представляет собой последовательность нулей и единиц, причём одни её участки кодируют команды, а другие — данные. Такая последовательность называется **машинным кодом**.

Язык ассемблера — это удобная для восприятия человеком форма записи машинных команд.

Между машинным кодом и кодом на языке ассемблера существует взаимно однозначное соответствие.

Программа, которая переводит текст с языка ассемблера в машинный код, называется **ассемблером**.

Для каждого процессора существует свой набор машинных команд, а следовательно, и свой ассемблер.

Язык программирования высокого уровня — это язык, разработанный для быстроты и удобства использования программистом.

Основная черта высокоуровневых языков — это **абстракция**, то есть введение смысловых конструкций, кратко описывающих такие структуры данных и операции над ними, описание которых на низкоуровневом языке или в машинном коде очень длинное и сложное для понимания.

Программа, написанная на языке высокого уровня, состоит из инструкций (операторов), при этом каждый оператор соответствует последовательности нескольких низкоуровневых инструкций (машинных команд).

Языки программирования высокого уровня принято разделять на группы, называемые **парадигмами**.

Парадигмы программирования — это совокупность идей и понятий, определяющих стиль написания программ.

В рамках каждой парадигмы существуют свои соглашения о том, какие подходы применяются при решении различных задач.

Языки программирования делятся на две группы: **декларативные** и **императивные**.

Декларативное программирование

В **декларативном** программировании задаётся спецификация решения задачи, описывается конечный результат, а не способ его достижения.

Декларативная программа состоит из ограничений и правил, из которых компьютер генерирует способ получения результата.

Примеры декларативных языков

- CSS
- HTML
- SQL

Императивное программирование

В императивном программировании программа представляет собой общее решение задачи, то есть последовательность команд, которые должен выполнить исполнитель.

Примеры императивных языков

- C/C++
- C#
- Python

В рамках императивной парадигмы существует несколько подходов к разбиению задач на подзадачи. Основные из них — это **процедурное** и **объектно-ориентированное** программирование.

Процедурное программирование — представление программы в виде совокупности процедур и функций.

Объектно-ориентированное программирование (ООП) — программа рассматривается как набор взаимодействующих друг с другом объектов, причём у каждого объекта есть свойства и поведение (методы).

Логическое программирование — программы основаны на математической логике.

Структурное программирование — методология разработки программы, в основе которой лежит представление программы в виде иерархической структуры блоков.
Каждый блок состоит из различных видов конструкций: последовательного исполнения, ветвления и циклов.

Язык C/C++

Язык C — это высокоуровневый императивный язык программирования. Стандарт языка (ANSI C) был утверждён в 1989 году Американским национальным институтом стандартов.
Язык был разработан Деннисом Ричи.

Язык C++ — высокоуровневый императивный язык, созданный Бьярне Страуструпом примерно в 1983 году. Является расширенной версией языка C, реализующей объектно-ориентированное программирование.
Первый стандарт (C++98) был окончательно принят в 1998 году.

Есть два способа выполнения (трансляции) программ компьютером: **компиляция** и **интерпретация**. Компилятор конвертирует всю программу в машинный код, а интерпретатор читает исходный код программы построчно и выполняет её строки последовательно.

Состав языка

Текст программы на языках C/C++ состоит из лексем, операторов и выражений.
Лексема — это минимальная единица языка программирования, имеющая самостоятельный смысл.
К ним относятся идентификаторы (имена программных объектов), ключевые зарезервированные слова (if, else), знаки операций (>, <, =), константы и разделители (скобки, запятые, точки, пробелы).

Оператор — это команда, математическое или логическое действие, выполняемое с данными (операндами).
Выражение — это последовательность операндов и операторов. Для их написания используются символы из алфавита языка.

Идентификаторы

Идентификатор — это имя программного объекта (переменной, функции, класса и т.д.).
Правила для идентификаторов:
- Допустимы только латинские буквы (a–z, A–Z), цифры (0–9) и символ подчеркивания _.
- Первым символом может быть только буква или _.
- Регистр букв имеет значение (например, `Variable` и `variable` — разные идентификаторы).

Ключевые слова — зарезервированные идентификаторы, имеющие специальное значение в языке.
Примеры: `if`, `else`, `while`, `for`, `int`, `return`.
Ключевые слова нельзя использовать в качестве пользовательских идентификаторов.

Константы

Константный литерал — это неименованная константа, представляющая неизменяемое значение.

Виды констант

1. **Целочисленные литералы**:
   - **Десятичные**: цифры от 0 до 9 (например, `123`).

- **Восьмеричные**: начинаются с `0`, цифры от 0 до 7 (например, `07`).
- **Шестнадцатеричные**: начинаются с `0x` или `0X`, цифры от 0 до 9 и буквы от A до F (например, `0x1A`).
- **Двоичные**: начинаются с `0b` или `0B`, цифры 0 и 1 (например, `0b1010`).

2. **Вещественные литералы**:
   - Форма с фиксированной точкой: содержат дробную часть, отделённую точкой (например, `3.14`).
   - Экспоненциальная форма: используют `e` или `E` для указания степени 10 (например, `1.278e2` = 127.8).
3. **Символьные литералы**:
   - Состоят из одного символа в одинарных кавычках (например, `'a'`).
   - Специальные символы: `\n` (новая строка), `\t` (табуляция), `\\` (обратная косая черта), `\'` (одинарная кавычка), `\"` (двойная кавычка).
4. **Строковые литералы**:
   - Последовательность символов в двойных кавычках (например, `"Hello, world!"`).
   - Могут включать специальные символы (например, `"Line1\nLine2"`).

Комментарии

Комментарий — это пояснение к исходному коду, игнорируемое компилятором.

- **Однострочные комментарии**: начинаются с `//` и продолжаются до конца строки.
  Пример: `// Это однострочный комментарий`.
- **Многострочные комментарии**: начинаются с `/*` и заканчиваются `*/`.
  Пример:

```
/* Это многострочный
   комментарий */
```

Переменные и именованные константы

Переменная — именованная область памяти для хранения изменяемых данных.

У переменной есть:

- **Тип**: определяет размер памяти, допустимые значения и операции.
- **Имя**: идентификатор переменной.
- **Значение**: данные, хранимые в переменной.

Именованная константа — именованная область памяти, значение которой нельзя изменить после инициализации.

Объявление и определение

- **Объявление (declaration)**: информирует компилятор о типе и классе памяти переменной или константы.
  Пример: `extern int x;`.
- **Определение (definition)**: выделяет память для переменной и может включать инициализацию.
  Пример: `int x = 5;`.

Формат определения

```
[класс памяти] [const] тип имя [= значение];
```

Примеры:

```
int i = 5;              // Переменная
int j{6};               // Инициализация в стиле C++11
const float PI = 3.14;  // Именованная константа
```

- Константы **обязательно** инициализируются при определении.

- Переменные можно не инициализировать, но это нежелательно (может привести к неопределённому поведению).

Множественная инициализация

Переменные одного типа можно определять в одной строке:

```
int a, b, c = 0;  // a и b не инициализированы, c = 0
```

Директива препроцессора

Именованные константы можно задавать с помощью `#define`:

```
#define N 5  // Константа N равна 5
```

Примечание: `#define` не создаёт типизированную константу, а выполняет текстовую замену.

Типы данных

Простые типы

1. **Целочисленные**:

   - `bool`: значения `true` или `false`, размер 1 байт.
   - `char`: знаковый или беззнаковый, диапазон -128..127 или 0..255, размер 1 байт.
   - `signed char`: -128..127, 1 байт.
   - `unsigned char`: 0..255, 1 байт.
   - `short int` (или `short`): -32,768..32,767, 2 байта.
   - `unsigned short int`: 0..65,535, 2 байта.
   - `int`: -2,147,483,648..2,147,483,647 (на 32-битных системах), 4 байта.
   - `unsigned int`: 0..4,294,967,295, 4 байта.
   - `long int` (или `long`): -2,147,483,648..2,147,483,647 (32-бит) или -$2^{63..2}$63-1 (64-бит), 4 или 8 байт.
   - `unsigned long int`: 0..4,294,967,295 (32-бит) или 0..2^64-1 (64-бит), 4 или 8 байт.
   - `long long int`: -$2^{63..2}$63-1, 8 байт.

- `unsigned long long int`: 0..2^64-1, 8 байт.

2. **С плавающей точкой**:
   - `float`: одинарная точность, ~6–7 знаков, 4 байта.
   - `double`: двойная точность, ~15–16 знаков, 8 байт.
   - `long double`: расширенная точность, зависит от платформы, обычно 10–16 байт.
3. **Пустой тип**:
   - `void`: используется для функций, не возвращающих значения, или указателей на неопределённый тип.

Составные типы

- Указатели (*)
- Массивы
- Структуры (`struct`)
- Объединения (`union`)
- Перечисления (`enum`)
- Функции
- Классы
- Ссылки (&)

Спецификаторы типов

- `signed`: знаковый тип (по умолчанию для `char`, `int`, `short`, `long`).
- `unsigned`: беззнаковый тип (положительные значения, увеличенный диапазон).
- `short`: уменьшенный размер для целочисленных типов.
- `long`: увеличенный размер для целочисленных или вещественных типов.

Определение размера типа

Оператор `sizeof` возвращает размер типа или переменной в байтах:

```
int a = 1;
std::cout << sizeof(a);  // Выводит 4 (на 32-битной системе)
```

Препроцессор

Препроцессор — текстовый процессор, выполняющий предварительную обработку исходного кода перед компиляцией.

Директивы препроцессора

Директивы начинаются с # и управляют обработкой кода. Они применяются к коду, расположенному ниже в файле.

- `#include`: вставляет содержимое другого файла (например, `#include <iostream>`).
- `#define`: определяет макрос или константу (например, `#define PI 3.14`).
- `#undef`: отменяет определение макроса.
- `#if`, `#elif`, `#else`, `#endif`: условная компиляция в зависимости от условий.
- `#ifdef`, `#ifndef`: проверяет, определён ли макрос.
- `#error`: генерирует сообщение об ошибке при компиляции.
- `#pragma`: задаёт специфические инструкции компилятору (зависит от компилятора).
- `#line`: изменяет номер строки и имя файла для отладки.
- `#import`: включает библиотеку типов (устаревшая, редко используется).
- `#using`: импортирует пространства имён (в основном в C++/CLI).

Пример условной компиляции:

```
#ifdef DEBUG
  std::cout << "Debug mode enabled";
#endif
```

Простейшие программы на языке C++

```
#include <iostream>
int main() {
    std::cout << "Hello, world!\n";
    return 0; // Программа завершилась успешно
}
```

Программа на C++ всегда содержит главную функцию **main**, которая является точкой входа. Тело функции заключается в **фигурные скобки** `{}`.
Текст внутри фигурных скобок следует форматировать с отступом в 4 пробела (или табуляцией) на каждом уровне вложенности для улучшения читаемости.

Пространство имён — это область программы, в которой определены имена (переменные, функции и т.д.). Имена, объявленные в одном пространстве имён, не конфликтуют с именами в других пространствах. Например, `std::cout` использует пространство имён `std` из стандартной библиотеки.

В директиве `#include` угловые скобки `<>` указывают, что файл ищется в стандартной директории компилятора (например, `/usr/include`). Двойные кавычки `""` указывают на поиск файла в текущей директории или указанном пути.

Каждый оператор в C++ должен заканчиваться точкой с запятой `;`.

Символы национальных алфавитов

Для поддержки русских символов в консоли на некоторых системах (например, Windows) требуется настройка кодовой страницы или локали. Вот несколько способов:

1. Использование `SetConsoleCP` для Windows:

```
#include <iostream>
#include <windows.h>
int main() {
    SetConsoleCP(1251);       // Кодовая страница для ввода (Windows-1251)
    SetConsoleOutputCP(1251); // Кодовая страница для вывода
    std::cout << "Привет, мир!\n";
    return 0;
}
```

2. Использование команды `chcp`:

```
#include <iostream>
#include <cstdlib>
int main() {
    system("chcp 1251"); // Установка кодовой страницы 1251
```

```
    std::cout << "Привет, мир!\n";
    return 0;
}
```

3. Использование локали:

```
#include <iostream>
#include <clocale>
int main() {
    setlocale(LC_ALL, "Russian"); // Установка русской локали
    std::cout << "Привет, мир!\n";
    return 0;
}
```

Ожидание ввода в конце программы

Чтобы консольное окно не закрывалось сразу после выполнения программы, можно использовать следующие методы:

1. Использование `system("pause")` (Windows):

```
#include <cstdlib>
int main() {
    std::cout << "Программа завершена.\n";
    system("pause"); // Ожидание нажатия клавиши
    return 0;
}
```

2. Использование `getchar()`:

```
#include <cstdio>
int main() {
    std::cout << "Программа завершена.\n";
    getchar(); // Ожидание ввода символа
    return 0;
}
```

3. Использование `std::cin.get()`:

```
#include <iostream>
int main() {
    std::cout << "Программа завершена.\n";
    std::cin.get(); // Ожидание нажатия Enter
```

```
    return 0;
}
```

Область видимости

Область видимости (scope) в C++ определяет, где переменная доступна. Область ограничивается блоком кода `{}`. Пример:

```
#include <iostream>
int a = 1; // Глобальная область видимости
int main() {
    int b = 3; // Локальная область видимости (в main)
    {
        int c = 5; // Вложенная область видимости
        std::cout << c << "\n"; // Выведет 5
    }
    // std::cout << c; // Ошибка: c не определена за пределами блока
    int c = 8; // Новая переменная c в области main
    std::cout << c << "\n"; // Выведет 8
    return 0;
}
```

Глобальные переменные по умолчанию инициализируются нулём.

Коллизия имён возникает, если в одной области видимости объявлены две переменные с одинаковым именем — это приведёт к ошибке компиляции. Однако во вложенных блоках можно переопределять имена:

```
#include <iostream>
int main() {
    int a = 7;
    int b = 8;
    // int a = 9; // Ошибка: повторное объявление a
    {
        int a = 9; // Допустимо: новая переменная a во вложенной области
        std::cout << a << "\n"; // Выведет 9
    }
    a = 9; // Допустимо: изменение значения существующей переменной
    std::cout << a << "\n"; // Выведет 9
    return 0;
}
```

Классы памяти

1. **Автоматические переменные** (стековые):

   Объявляются внутри функций без спецификаторов. Хранятся в стеке, их время жизни ограничено блоком кода. В современных стандартах C++ ключевое слово `auto` не используется для этого, а определяет автоматический вывод типа.

2. **Динамическая память**:

   Переменные создаются в куче с помощью `new` и освобождаются с помощью `delete`. Доступны везде, где известен их адрес. Память нужно освобождать вручную, иначе она сохраняется до завершения программы.

3. **Статическая память**:

   Статические переменные существуют в течение всей работы программы и инициализируются нулём. В функциях сохраняют значение между вызовами:

```
#include <iostream>
void print_next() {
    static int i = 0; // Статическая переменная
    i += 1;
    std::cout << i << "\n";
}
int main() {
    print_next(); // Выведет 1
    print_next(); // Выведет 2
    print_next(); // Выведет 3
    return 0;
}
```

4. **Регистровые переменные**:

   Объявляются с ключевым словом `register`. Компилятор может разместить их в регистрах процессора для ускорения, но это лишь рекомендация, и компилятор может её игнорировать.

Выражения и математические функции

Выражения в C++ состоят из операций: арифметических, логических, битовых и других.

- **Унарные операции** работают с одним операндом.

- **Бинарные операции** работают с двумя операндами.

- **Тернарная операция** (единственная в C++) работает с тремя операндами.

Операция присваивания

Синтаксис: `операнд1 = операнд2;`

Сначала вычисляется правый операнд, затем его значение присваивается левому.

Множественное присваивание:

```
int a, b, c;
a = b = c = 0; // Все переменные станут равны 0
```

Унарные операции

1. **Инкремент (++)**: Увеличивает значение на 1.

   - **Преинкремент** (`++x`): Увеличивает значение до использования.

   - **Постинкремент** (`x++`): Использует текущее значение, затем увеличивает.

     Пример:

```
int x = 5;
std::cout << ++x << "\n"; // Выведет 6 (x увеличивается до вывода)
std::cout << x++ << "\n"; // Выведет 6 (x увеличивается после вывода)
std::cout << x << "\n";   // Выведет 7
```

2. **Декремент (`--`)**: Уменьшает значение на 1.

   - **Предекремент** (`--x`): Уменьшает до использования.

   - **Постдекремент** (`x--`): Уменьшает после использования.

3. **Логическое отрицание (`!`)**: Инвертирует логическое значение (`true` → `false`, `false` → `true`).

```
bool flag = true;
std::cout << !flag << "\n"; // Выведет 0 (false)
```

4. **Поразрядная инверсия (~)**: Инвертирует все биты числа.

```
int x = 5; // 00000101 в двоичной системе
std::cout << ~x << "\n"; // Выведет -6 (11111010 в двоичной системе)
```

5. **Унарный плюс (+)**: Не изменяет значение, редко используется.

```
int x = 5;
std::cout << +x << "\n"; // Выведет 5
```

6. **Унарный минус (-)**: Меняет знак числа.

```
int x = 5;
std::cout << -x << "\n"; // Выведет -5
```

Бинарные операции

1. **Сложение (+)**: Складывает два операнда.

2. **Вычитание (-)**: Вычитает второй операнд из первого.

3. **Умножение (*)**: Умножает два операнда.

4. **Деление (/)**: Делит первый операнд на второй. Для целых чисел результат целочисленный.

5. **Остаток от деления (%)**: Возвращает остаток от деления.

6. **Битовый сдвиг вправо (>>)**: Сдвигает биты первого операнда вправо на количество позиций, заданное вторым операндом.

7. **Битовый сдвиг влево (<<)**: Сдвигает биты влево.

8. **Операторы сравнения**: >, <, >=, <=, ==, `!=`. Возвращают `true` или `false`.

9. **Поразрядная конъюнкция (&)**: Битовое “И”.

10. **Поразрядное исключающее ИЛИ (^)**: Битовое “XOR”.

11. **Поразрядное ИЛИ (|)**: Битовое “ИЛИ”.

12. **Логическое И (&&)**: Возвращает `true`, если оба операнда истинны.

13. **Логическое ИЛИ (||)**: Возвращает `true`, если хотя бы один операнд истинен.

14. **Операции присваивания**: =, +=, -=, *=, /=, %=, >>=, <<=, &=, |=, ^=.

    Пример:

```
int a = 10, b = 3;
std::cout << (a + b) << "\n"; // Выведет 13
std::cout << (a % b) << "\n"; // Выведет 1
std::cout << (a > b) << "\n"; // Выведет 1 (true)
```

Тернарный оператор

Синтаксис: `условие ? выражение1 : выражение2`

Если условие истинно, возвращается `выражение1`, иначе — `выражение2`.

Пример:

```
int x = 5, y = 10;
int max = (x > y) ? x : y; // max = 10
std::cout << max << "\n";
```

Преобразование типов

В C++ существует два вида преобразования типов: **неявное преобразование** и **явное преобразование** (приведение типов).

Неявное преобразование

При выполнении операции присваивания значение, получившееся при вычислении правого операнда, автоматически приводится к типу левого операнда. Это может привести к потере данных, если типы несовместимы.

Пример:

```
int a = 10.7; // Неявное преобразование float к int, результат: a = 10
float b = 5;  // Неявное преобразование int к float, результат: b = 5.0
```

Явное преобразование типов

Позволяет явно указать требуемый тип, чтобы избежать ошибок неявного преобразования. Используются операторы приведения, такие как `static_cast`, `dynamic_cast`, `const_cast`, или C-стиль `(тип)`.

Пример:

```
double x = 15.99;
int y = static_cast<int>(x); // Явное преобразование double к int, результат: y = 15
```

Операции битового сдвига

Сдвигают двоичное представление левого операнда на количество разрядов, заданное правым операндом. Сдвиг может быть влево (<<) или вправо (>>).

Пример сдвига влево:

```
int a = 5; // 0b00000101
int b = a << 1; // Сдвиг влево на 1, результат: 0b00001010 (10 в десятичной)
```

Пример сдвига вправо:

```
int a = 10; // 0b00001010
int b = a >> 1; // Сдвиг вправо на 1, результат: 0b00000101 (5 в десятичной)
```

Ввод и вывод с консоли

По умолчанию C++ не содержит встроенных средств для ввода-вывода с консоли. Эти средства предоставляются библиотекой `<iostream>`, являющейся частью стандартной библиотеки STL. В ней определены типы `istream` (поток ввода) и `ostream` (поток вывода).

Поток — последовательность символов, которая записывается на устройство ввода-вывода или считывается с него. Для вывода на консоль используется объект `cout`, для ввода — объект `cin`.

Пример:

```
#include <iostream>
using namespace std;

int main() {
    int x;
    cout << "Введите число: ";
    cin >> x;
    cout << "Вы ввели: " << x << endl;
    return 0;
}
```

Манипуляторы форматирования для формата выводимых чисел

Манипуляторы определены в заголовочном файле `<iomanip>`.

1. `setw(n)`: задаёт ширину поля для вывода числа (удобно для таблиц).
2. `setprecision(n)`: задаёт количество знаков после десятичной точки для чисел с плавающей точкой.
3. `fixed`: устанавливает формат чисел с фиксированной точкой.
4. `scientific`: устанавливает экспоненциальный формат вывода чисел.

Пример:

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double num = 123.456789;
    cout << setw(10) << num << endl; // Ширина поля 10
    cout << fixed << setprecision(2) << num << endl; // 123.46
    cout << scientific << setprecision(3) << num << endl; // 1.235e+02
    return 0;
}
```

Условные операторы

Вычислительный процесс называется **разветвляющимся**, если в зависимости от выполнения некоторого условия он реализуется по одному из нескольких направлений. Для этого используются операторы `if` и `switch`.

Оператор if

Синтаксис:

```
if (условие) {
    // тело условия
} else if (другое_условие) {
    // тело другого условия
} else {
    // тело, если условия выше не выполнились
}
```

Наличие `else` необязательно.

Пример:

```
int x = 10;
if (x > 0) {
    cout << "Число положительное" << endl;
} else if (x < 0) {
    cout << "Число отрицательное" << endl;
} else {
    cout << "Число равно нулю" << endl;
}
```

Оператор switch (оператор ветвления)

Оператор `switch` используется для выбора одного из множества вариантов выполнения на основе значения выражения. Подходит для случаев, когда нужно сравнить переменную с несколькими константами.

Синтаксис:

```
switch (выражение) {
    case значение1:
        // код для значение1
        break;
    case значение2:
        // код для значение2
        break;
    default:
        // код, если ни одно значение не подошло
}
```

Пример:

```
#include <iostream>
using namespace std;

int main() {
```

```
    int day = 3;
    switch (day) {
        case 1:
            cout << "Понедельник" << endl;
            break;
        case 2:
            cout << "Вторник" << endl;
            break;
        case 3:
            cout << "Среда" << endl;
            break;
        default:
            cout << "Другой день" << endl;
    }
    return 0;
}
```

Статическая типизация и неявное преобразования типов в C++

Статическая типизация

- C++ — **статически типизированный** язык.
- Тип переменной фиксируется при объявлении и не может быть изменён.
- Переменная может хранить только значения своего типа.
- Для присваивания значений других типов используются **преобразования типов**.
- Некоторые преобразования выполняются **неявно** (автоматически), но не всегда.

Пример ошибки при неявном преобразовании

```
unsigned int age = -25;  // Ошибка компиляции: unsigned не может хранить отрицательные значения
```

Неявные (имплицитные) преобразования

1. Присваивание bool

- Значение → `false`, если 0.
- Во всех остальных случаях → `true`.

```
bool a = 1;      // true
bool b = 0;      // false
bool c = 'g';    // true
bool d = 3.4;    // true
```

2. Присваивание числу/символу значения bool

- `true` → 1
- `false` → 0

```
int c = true;    // 1
double d = false;// 0
```

3. Целому числу — дробное

- Дробная часть **отбрасывается** (усечение к нулю).

```
int a = 3.4;     // 3
int b = 3.6;     // 3
```

4. Типу с плавающей точкой — целое

- Если целое не помещается → потеря точности (усечение).

```
float a = 35005;                    // OK
double b = 3500500000033;            // ≈ 3.5005e+12 (потеря точности)
```

5. Unsigned-типу — значение вне диапазона

- Результат: **остаток от деления по модулю** (2^n, где n — разрядность).

```
unsigned char a = -5;       // 251 (255 - 4)
unsigned short b = -3500;   // 62036
unsigned int c = -50000000; // 4244967296
```

6. Signed-типу — значение вне диапазона

- **Неопределённое поведение** (undefined behavior): может работать корректно или некорректно.

Преобразования в арифметических операциях (usual arithmetic conversions)

- Операнды приводятся к **общему типу**.
- Выбирается тип с **большим приоритетом** (promotion).
- Иерархия типов (от высшего к низшему):
  1. `long double`
  2. `double`
  3. `float`
  4. `unsigned long long`
  5. `long long`
  6. `unsigned long`
  7. `long`
  8. `unsigned int`
  9. `int`
- Типы `char`, `signed char`, `unsigned char`, `short`, `unsigned short` → как минимум к `int`.

Примеры

```
double sum = 100.2;
int hours = 8;
double revenuePerHour = sum / hours;  // hours преобразуется к double
```

Опасный пример

```
int n = 5;
unsigned int x = 8;
std::cout << n - x;  // Результат: 4294967293 (n преобразуется к unsigned int)
```

Безопасные и опасные преобразования

Безопасные (без потери информации)

- От меньшей разрядности к большей:
  - `bool → char → short → int → double → long double`
  - `bool → char → short → int → long → long long`
  - `unsigned char → unsigned short → unsigned int → unsigned long`
  - `float → double → long double`

```
short a = 'g';
double c = 10;      // int → double
double e = 3.4f;    // float → double
```

Опасные (возможна потеря точности/данных)

- От большей разрядности к меньшей или вне диапазона.
- Часто выдаётся предупреждение компилятора, но код компилируется.

```
unsigned int a = -25;      // 4294967271
unsigned short b = -3500;  // 62036
```

> **Важно**: значение — набор битов, интерпретация зависит от типа.

Рекурсивные функции

- Функция вызывает сама себя.
- Обязательно должна иметь **терминальную ветвь** (условие завершения).
- Минусы: расход памяти и времени на стек → риск переполнения стека.
- Рекомендация: по возможности заменять рекурсию циклом.

Перегрузка функций (function overloading)

- Несколько функций с одинаковым именем, но разными параметрами (по количеству и/или типам).
- Компилятор выбирает подходящую по типам аргументов.
- При отсутствии точного совпадения — применяются стандартные преобразования.
- Если несколько вариантов подходят → **неоднозначность** → ошибка компиляции (нужно явное приведение).

Значения параметров по умолчанию

- Можно задать для последних параметров.
- Если аргумент пропущен — используется значение по умолчанию.

Стек и вызовы функций

- Стек (LIFO) хранит:
  - Параметры функции.
  - Адрес возврата.
- При вложенных вызовах — параметры и адреса сохраняются в стеке.

Встраиваемые функции (inline)

- Предложение компилятору подставить код функции напрямую в место вызова.
- Экономит время (нет переходов и работы со стеком).
- Подходит для небольших функций.
- Решение о встраивании принимает компилятор.
- Рекурсивные функции **не встраиваются**.

Способы передачи параметров

1. По значению

- В стек копируется **значение**.
- Изменения внутри функции не влияют на оригинал.
- Подходит для небольших данных.

2. По адресу (указатели)

- В стек копируется **адрес**.
- Позволяет изменять оригинал через разыменование *.
- Указатель: `тип *имя;`
- Операции: & (адрес), * (разыменование).

3. По ссылке (рекомендуется в C++)

- В стек копируется **адрес**, но синтаксис как с обычной переменной.
- Параметр: `тип &имя`
- Позволяет изменять оригинал и возвращать несколько значений.
- Эффективно для больших объектов (без копирования).
- **Отличия от указателей**:
  - Ссылку нельзя переприсвоить.
  - Нет арифметики ссылок.
  - Нет нулевой ссылки.
  - Ссылка всегда инициализирована.
  - Нет массивов ссылок.

Преобразования и ссылки

- При передаче **по значению** допускаются неявные преобразования (даже с потерей точности).
- При передаче **по ссылке** неявные преобразования **запрещены** → исключает нежелательные потери точности.

```
void printVal(int n);      // OK: double → int
void printRef(int &n);     // Ошибка с double
```

Заполнение массива случайными числами

Для генерации случайных чисел будет использоваться датчик псевдослучайных чисел из заголовка **cstdlib**. Функция rand генерирует псевдослучайные числа от 0 до константы **RAND_MAX**, которая в зависимости от реализации может иметь разные значения, но не меньше 32767. Перед использованием функции rand необходимо инициализировать датчик случайных чисел начальным значением - **зерном**, для чего используется функция **srand**, принимающая число типа unsigned long int. Для различных значений при разных запусках программы начальное значение можно задать с помощью системного таймера, использовав функцию **time** из библиотеки **ctime**. `srand(time(0))` Эта функция вернёт количество секунд, прошедшее с 01.01.1970. 1. Для целых чисел int t num = min rand() % (max - min + 1); 2. для действительных чисел float num = 1.0 * rand() / RAND_MAX) * (max - min) + min;

Многомерные массивы

Двумерные массивы

Двумерный массив или матрица объявляется как массив массивов. тип имя_массива[][] `int matrix[2][3];` Индексы также пронумированы с нуля. `int matrix[2][3] = {{2, 3, 5}, {7, 8, 9}};`

Трехмерный массив

```
int matrix[5][5][5];
```

Заполнение двумерных массивов

```
int matrix[2][3];
for (int i = 0; int < 2; i++) {
    for (int j = 0; j < 3; j++) {
        std::cout << "Введите элемент [" << i << ", " << j << "]\n";
        std::cin >> matrix[i][j];
    }
}
```

```
## Подпрограммы
```

При разработке программ часто необходимо выполнять одни и те же операции. Для исключения повторений одинакового

Такая автономная часть программы, реализующая определенный алгоритм и оформленная в виде отдельной синтаксическ

В языке c++ подпрограммами называются функции.

Функция - это именованная последовательность описаний и операторов, выполняющая какое-

либо законченное действие. Функция может принимать параметры и возвращать значения.

В программе главная функция main.

Любая другая функция должна быть объявлена и определена.

Объявление функции задаёт её имя, тип параметров и тип возвращаемого значения. В **определении** функции кро

Тело функции - последовательность объявлений и операторов, из которых состоит сама функция. Тело функции пом

Объявление функции должно быть раньше её определения.

Функция объявляется так:

тип_возвращаемого_значения имя_функции(параметр1 тип_параметра1, параметр_н тип_параметра_н);

а определяется так:

тип_возвращаемого_значения имя_функции(тип_параметра1 параметр1, тип_параметра_н параметр_н) {

    тело

}

Тип возвращаемого значения может быть любым кроме массива и функции, но может являться указателм на массив или фу

Если функция не возвращает значения, тип возвращаемого значения будет** void**.

Передача результата из функции в вызывающую программу осуществляется с помощью оператора **return**.

В определении функции указываются формальные параметры, а при вызове -фактические. Имена формальных и фактическ

перегрузка функций.

Пример функции

```
#include <cmath>
long len(float x, float y)  {
    return sqrt(x*x + y*y);
}
```

Параметры функций

Передача параметров: по значению, по указателю, по ссылке

- **По значению**: создаётся копия аргумента; изменения внутри функции не влияют на оригинал.

  Пример:

  ```cpp
  void inc(int x) { x++; }  // оригинал не изменится
  ```

- **По указателю**: передаётся адрес; через разыменование можно изменить оригинал.

  Пример:

```
void inc(int* p) { (*p)++; }  // оригинал изменится
```

- **По ссылке**: передаётся псевдоним переменной; изменения затрагивают оригинал (без *).

  Пример:

```
void inc(int& r) { r++; }  // оригинал изменится
```

Константные параметры

Параметры могут быть `const` — тогда их значение нельзя изменить внутри функции.

Пример: `const int` для возведения в степень 2

```
int square(const int x) {
    // x = 5;  // ошибка: изменение const
    return x * x;
}
```

- При **объявлении** функции `const` для параметров по значению можно опустить.

- Для **указателей/ссылок** `const` **обязателен** и в объявлении, и в определении.

К `const`-параметру можно передать: - константу (`5`, `const int c = 10`); - переменную (значение скопируется/ссылка будет на `const`).

Константу можно передать в не-`const` параметр (копируется).

Константные параметры-ссылки

```
void print(const std::string& s) {
    // s += "!";  // ошибка
    std::cout << s;
}
```

Значение менять нельзя, но ссылка эффективна (без копии).

Константные параметры-указатели

```
void sum(int a, int b, const int* c) {
    // *c = a + b;  // ошибка компиляции
    std::cout << *c;
}
```

- Нельзя менять значение по указателю.

- Сам указатель (адрес) можно менять: `c++;`.

Константный указатель на константу — неизменяемы и адрес, и значение:

```
void safe(const int* const p) {
    // *p = 5;  // ошибка
    // p++;     // ошибка
}
```

Указатель на `const` нельзя передать в параметр-указатель без `const`.

Массивы в параметрах функции

Массив **вырождается в указатель** на первый элемент:

```
void print_array(int* a);        // эквивалентно
void print_array(int a[]);       // синтаксический сахар
void print_array(int a[10]);     // размер игнорируется
```

Размер массива **не передаётся** — нужно передавать отдельно:

```
void print_array(const int* a, size_t size) {
    for (size_t i = 0; i < size; ++i) {
        std::cout << a[i] << ' ';
    }
}
```

Константные ссылки на массивы

Передача по ссылке сохраняет размер и предотвращает вырождение:

```
void print_array(const int (&a)[5]) {  // только для массивов из 5 элементов
    for (int x : a) std::cout << x << ' ';
}
```

Преимущества: - Не копируется адрес. - Работа с оригинальным массивом. - Гарантия размера на этапе компиляции. - Запрет изменения (с `const`).

Без `const` элементы можно менять → используй `const`, если изменение не нужно.

Параметры функции `main`

```
int main(int argc, char* argv[])  // или char** argv
```

- `argc` — число аргументов **включая имя программы** (`./prog a b` → `argc == 3`).
- `argv[0]` — имя программы.

- `argv[1..argc-1]` — переданные аргументы как C-строки.

  Пример:

```
int main(int argc, char* argv[]) {
    for (int i = 0; i < argc; ++i) {
        std::cout << argv[i] << '\n';
    }
}
```

Указатели

Для данных произвольного размера (массивы, структуры) используется **динамическая память** — **куча** (heap), выделяемая ОС.

Переменные в куче — **динамические**.

Указатель — переменная, хранящая **адрес** в памяти.

Синтаксис:

```
тип* имя_указателя;   // рекомендуется: звезда рядом с именем
// или: тип *имя;     // допустимо, но менее читаемо
```

- Размер указателя **фиксирован**: 4 байта (32-bit) или 8 байт (64-bit), **независимо от типа данных**.
- Указатель знает **тип** данных, на которые указывает → корректная арифметика и разыменование.

Выделение и освобождение динамической памяти

Ранее указатели использовались для хранения адресов обычных именованных переменных, но основное назначение указателей — работа с безымянными объектами, размещёнными в динамической памяти (куче).

Для выделения динамической памяти: - в C++ используется оператор **new** (и **new[]** для массивов) - в C — функции **malloc**, **calloc**, **realloc**

Примеры в C++:

```
int* pn = new int;              // безымянная переменная типа int
int* arr = new int[10];         // динамический массив из 10 int
double* pt = new double(3.5);   // с инициализацией
double* mas = new double[5]{};  // все элементы = 0.0
```

После окончания работы с динамической памятью её **обязательно** нужно освободить, иначе возникнет утечка памяти. Память будет занята до завершения программы (ОС освободит её автоматически только при выходе).

Освобождение в C++:

```
int* pn = new int;
delete pn;       // освобождаем одиночный объект
pn = nullptr;    // хорошая практика — обнулить указатель

int* arr = new int[10];
delete[] arr;    // обязательно с [] для массива!
arr = nullptr;
```

Важно: без `[]` при `delete` для массива поведение неопределённое.

Двумерные динамические массивы

Для создания настоящего двумерного динамического массива произвольного размера обычно используют иерархическую структуру: массив указателей на строки, где каждая строка — свой динамический массив.

```
int rows = 3;
int cols = 5;

int** matrix = new int*[rows];              // массив указателей на строки
for (int i = 0; i < rows; i++) {
    matrix[i] = new int[cols]{};            // каждая строка — свой массив
}

// использование: matrix[i][j]

// освобождение в обратном порядке
for (int i = 0; i < rows; i++) {
    delete[] matrix[i];                     // сначала строки
}
delete[] matrix;                            // потом массив указателей
matrix = nullptr;
```

Преимущества иерархического подхода

- Можно делать строки разной длины (jagged array)
- Логически ближе к «массиву массивов»

Недостатки

- Невозможна обычная арифметика указателей по двум измерениям
- Дополнительные затраты памяти на массив указателей
- Двухэтапное выделение/освобождение

Альтернатива: «плоский» одномерный массив

Можно представить двумерный массив как один большой одномерный и эмулировать индексацию:

```
int* flat = new int[rows * cols]{};

// доступ: flat[i * cols + j] вместо matrix[i][j]

delete[] flat;  // освобождение одним вызовом
```

Этот способ экономит память и позволяет использовать арифметику указателей, но теряется возможность строк разной длины.

Структуры в C/C++

- **Структура** — это тип данных, состоящий из **фиксированного числа именованных элементов** (полей).
- Типы полей могут быть различными.
- Элементы структуры называются **полями**.

Назначение структур

- Группировка связанных данных, которые нужно хранить вместе (особенно в массивах).

  Пример: массив точек с координатами `x` и `y`.
- Возврат из функции нескольких связанных значений.

  Пример: функция возвращает дату (день, месяц, год).

Объявление типа структуры

```
struct <имя_структуры> {
    <тип_поля1> <имя_поля1>;
    ...
    <тип_поляN> <имя_поляN>;
};
```

- `<тип_поля>` — любой предопределённый или ранее объявленный тип (включая массивы, указатели).
- Нельзя использовать саму структуру внутри своего объявления (кроме указателя на себя).
- `struct <имя_структуры>` становится новым типом данных (должен быть уникальным).

Видимость типа

- Объявлена **внутри функции/блока** → видна только в этом блоке.
- Объявлена **вне функций** → видна во всём файле.

Пример объявления типа `student`

```
struct student {
    char FIO[41];       // ФИО (до 40 символов + '\0')
    int course;         // номер курса
    float avg_rating;   // средний рейтинг
};
```

Объявление переменных типа структура

- Единичная переменная:

  `[struct] <имя_структуры> <имя_переменной>;`

- Массив структур:

  `[struct] <имя_структуры> <имя_массива>[<размер>];`

- Указатель на структуру:

  `[struct] <имя_структуры> *<имя_переменной>;`

Пример

```
struct student students[25];   // массив из 25 студентов
struct student *topStudent;    // указатель на лучшего студента

// После объявления типа можно опускать struct
student students[25];
student *topStudent;
```

Совместное объявление типа и переменной (анонимная структура)

```
struct [<имя_структуры>] {
    // поля
} <имя_переменной1>, <имя_переменной2>, ...;
```

- Если имя структуры не указано — структура **анонимная** (используется только для объявленных переменных).

Пример анонимной структуры

```
struct {
    char FIO[41];
    int course;
    float avg_rating;
} students[25], *topStudents;
```

Вложенные структуры

- Одна структура может содержать поле другого структурного типа.
- Вложенные структуры часто бывают анонимными.

Пример: расширенная структура `student_ext`

```
struct student_ext {
    char FIO[41];
    int course;
    float avg_rating;
```

```
    struct {
        char name[36];  // название предмета
        int grade;      // оценка
    } subjects[2];      // массив из 2 предметов
};
```

Инициализация структур

```
struct <имя_структуры> <имя_переменной> = {
    <значение_поля1>,
    <значение_поля2>,
    ...
    <значение_поляN>
};
```

Примеры

```
struct student Iam = {"Иванов И.И.", 1, 99.9};


struct student_ext other = {
    "Петров А.П.", 1, 21.0,
    {                   // массив subjects
        {"Физика", 12},
        {"Программирование", 30}
    }
};
```

Операции над структурами

- `=` — присваивание (поэлементное копирование, только для структур одного типа).
- `.` — обращение к полю структуры.
- `->` — обращение к полю через указатель.

Обращение к полям

- `<переменная_структура>.<имя_поля>`
- `<указатель_на_структуру>-><имя_поля>`

  Оба варианта возвращают **l-value** (можно присваивать).

Общий случай

- `<выражение_возвращающее_структуру>.<имя_поля>`
- `<выражение_возвращающее_указатель>-><имя_поля>`

Примеры

```
strcpy(students[0].FIO, "Петров А.П.");          // элемент массива

Iam.subjects[1].grade = 99;                      // вложенная структура

getStudent(160, i)->course += 1;                 // через функцию

(*topStudent).FIO;                               // эквивалент topStudent->FIO
```

Примеры выполнения заданий

```
// Увеличить курс студента Iam на 1
Iam.course++;

// Вывести ФИО студента Iam
puts(Iam.FIO);

// Задать первый предмет у other
strcpy(other.subjects[0].name, "Философия");
other.subjects[0].grade = 100;

// Вывести данные лучшего студента (через ->)
printf("%s %f\n", topStudent->FIO, topStudent->avg_rating);

// То же самое через .
printf("%s %f\n", (*topStudent).FIO, (*topStudent).avg_rating);
```

Символы и строки в C++

Символы

- **Символ** — элементарная единица, набор которых несёт смысл.
- **Символьная константа** — целочисленное значение типа `int`, представленное символом в одинарных кавычках, например `'a'`.
- Символы и их коды определены в таблице ASCII.

Строки в стиле C (C-строки)

- Представляются как **массивы типа `char`**, заканчивающиеся **нуль-терминатором** `\0`.
- `\0` — обязательный завершающий символ (нулевой байт).
- Отсутствие или удаление `\0` приводит к **недетерминированному поведению** → C-строки считаются **небезопасными**.
- Строковые литералы заключаются в **двойные кавычки**: `"abcdef"`.
- При объявлении массива необходимо учитывать место под `\0` (компилятор добавляет его автоматически).

Объявление и инициализация

```
char str[10] = "abcdefghf";  // 9 символов + \0 → требуется 10 байт
char str[] = "abc";          // размер определяется автоматически, добавляется \0
```

- Без указания размера обязательно нужна инициализация.
- Строки в динамической памяти:
  - Описывается указатель `char*`.
  - Выделяется память, но **инициализировать напрямую нельзя**.
  - `const char* p = "Привет!";` → создаёт **строковую константу** (неизменяемую).

Ввод/вывод C-строк

- `gets()` — **устаревшая и небезопасная** (не контролирует размер буфера, может переполнить).
- `gets_s()` — безопасная версия с обработчиком недопустимых параметров.
- `cin` — считывает до первого пробела.
- `cin.getline(buf, size)` — считывает включая пробелы, до `\n` или указанного размера (включая `\0`).
- `cin.getline(buf, size, delim)` — третий параметр — символ-разделитель (не включается в строку).

Двухмерные массивы символов (массив строк)

```
const int max_length = 50;
char langs[][max_length] = {
    "C++", "C#", "Python", "Java",
```

```
    "Kotlin", "Go", "Dart", "PHP"
};
```

- Каждый элемент — отдельная C-строка.
- Неиспользуемые позиции в строках заполняются `\0` по умолчанию.

Строки типа `std::string`

- Определяется в заголовке `<string>`.
- Более безопасная альтернатива C-строкам.
- Внутренне хранит массив `char` с `\0`, но предоставляет удобный и безопасный интерфейс.

Определение и инициализация

```
std::string s;                    // пустая строка
std::string message = "Привет";   // копия литерала
std::string s(10, 'a');           // "aaaaaaaaaa"
std::string s2(other_string);     // копия другой строки
std::string s3(str, pos, n);      // часть строки
```

Ввод/вывод

- `cout << str;` / `cin >> str;` — `cin` до первого пробела.
- `std::getline(std::cin, str);` — считывает всю строку включая пробелы до `\n`.

Размер и проверка пустоты

```
str.length()  // или str.size() — количество символов (без \0)
str.empty()   // true, если строка пустая
```

Объединение строк

```
std::string result = str1 + str2;
```

- Нельзя напрямую складывать два строковых литерала: `"a" + "b"` → ошибка.
- Решения:
  - `"a" "b"` (автоматическое объединение литералов).
  - Использовать суффикс `s` (с `using namespace std::string_literals;`):

    ```
    auto s = "Привет, "s + "мир!"s;
    ```

Raw-строковые литералы

- Префикс `R"(..." )"`
- Позволяют писать специальные символы (`\`, `"`, `\n`, `\t`) без экранирования.

```
std::string text = R"(Имя:  Вася
Возраст:    38
Символы: \ " )";
```

Преобразование типов

В строку

```
std::to_string(value);  // int, float, double и т.д. → string
```

Из строки в число

Функции из `<string>`:

- `std::stoi()` → `int`
- `std::stol()` → `long`
- `std::stoll()` → `long long`
- `std::stoul()`, `std::stoull()` → unsigned
- `std::stof()` → `float`
- `std::stod()` → `double`
- `std::stold()` → `long double`

В C-строку

```
const char* c_str = str.c_str();  // указатель на внутренний буфер (const)
```

Сравнение строк в C++

К строкам `std::string` можно применять операции сравнения: `==`, `!=`, `>`, `<`, `>=`, `<=`.

Строки сравниваются **посимвольно** в **лексикографическом порядке** (словарном порядке) до первого различия или до конца одной из строк.

- Если символы различаются, сравниваются их коды (ASCII/Unicode).
- Если одна строка является префиксом другой, более короткая считается меньшей.

```
std::string a = "apple";
std::string b = "apricot";
std::string c = "apple";
```

```
std::cout << (a < b)  << std::endl;  // true (p < r)
std::cout << (a == c) << std::endl;  // true
std::cout << (a > "app") << std::endl; // true ("apple" > "app")
```

Метод `compare()`

Метод `std::string::compare()` возвращает целое число:

- `< 0` — если вызывающая строка лексикографически меньше
- `== 0` — если строки равны
- `> 0` — если вызывающая строка больше

```
std::string a = "123";
std::string b = "124";
std::cout << a.compare(b) << std::endl; // -1 (a < b)

std::string c = "123";
std::cout << c.compare("123") << std::endl; // 0 (равны)

std::cout << b.compare(a) << std::endl; // +1 (b > a)
```

Получение подстроки: `substr()`

Метод `substr(pos, len)` возвращает подстроку, начиная с позиции `pos` длиной `len` символов.

- Если `len` выходит за пределы строки → берётся до конца.
- Если указан только `pos` → берётся до конца.
- Если `pos` ≥ `size()` → выбрасывается исключение `std::out_of_range`.

```
std::string s = "Hello, world!";
std::cout << s.substr(7, 5) << std::endl; // "world"
std::cout << s.substr(0, 5) << std::endl; // "Hello"
std::cout << s.substr(7)    << std::endl; // "world!"
```

Поиск подстроки: `find()` и `rfind()`

Методы возвращают позицию первого/последнего вхождения подстроки (или символа) или `std::string::npos`, если ничего не найдено.

`npos` — это максимальное значение `size_t` (обычно `-1` в виде unsigned).

```
std::string s = "Hello, world! Hello again!";
size_t pos1 = s.find("Hello");          // 0
size_t pos2 = s.find("Hello", 1);       // 14 (поиск начиная с позиции 1)
size_t pos3 = s.find('w');              // 7
```

```
size_t pos4 = s.rfind("Hello");         // 14 (последнее вхождение)
size_t pos5 = s.find("xyz");            // npos (не найдено)

if (pos5 == std::string::npos) {
    std::cout << "Не найдено" << std::endl;
}
```

Модификация строки

- `append()` — добавляет строку в конец.
- `insert(pos, str)` — вставляет строку `str` перед позицией `pos`.
- `replace(pos, len, str)` — заменяет `len` символов начиная с `pos` на `str`.
- `erase(pos, len)` — удаляет `len` символов начиная с `pos` (или до конца).

```
std::string s = "Hello world!";

// append
s.append(" C++");           // "Hello world! C++"

// insert
s.insert(5, ",");           // "Hello, world! C++"

// replace
s.replace(6, 6, "friend");  // "Hello,friend! C++"

// erase
s.erase(5, 1);              // "Hellofriend! C++"
```

Проверка символов

Для проверки **отдельных** символов используются функции из `<cctype>`:

Функция	Проверяет, является ли символ…
`isupper(c)`	заглавной буквой (A–Z)
`islower(c)`	строчной буквой (a–z)
`isdigit(c)`	цифрой (0–9)
`isxdigit(c)`	шестнадцатеричной цифрой (0–9A–F)
`isalnum(c)`	буквой или цифрой (a–zA–Z0–9)
`isspace(c)`	пробельным символом (`пробел, \n, \t, \r, \f`)
`isblank(c)`	пробелом или табуляцией
`ispunct(c)`	знаком препинания
`isprint(c)`	печатным (видимым)

Функция	Проверяет, является ли символ…
`iscontrol(c)`	управляющим символом

```
#include <cctype>


std::string s = "A1b2C3";
for (char c : s) {
    if (isupper(c)) std::cout << c << " — заглавная\n";
    if (isdigit(c)) std::cout << c << " — цифра\n";
}
```

Функции `tolower(c)` и `toupper(c)` (из `<cctype>`) преобразуют символ в нижний/верхний регистр.

Низкий уровень

Память программы

Операционная система для каждой запущенной программы выделяет **виртуальное адресное пространство**. Это позволяет программам выполняться в изолированных «песочницах», не мешая друг другу и не имея прямого доступа к памяти других процессов.

Схема виртуальной памяти программы (от нижних адресов к верхним)

1. **Program text (текст программы)**
   - Содержит машинный код исполняемых инструкций программы.
   - Обычно размещается в нижней части адресного пространства.
   - Только для чтения (read-only), защищена от записи.
2. **Initialized data (инициализированные данные)**
   - Глобальные переменные и статические локальные переменные, которым явно присвоены начальные значения.
   - Пример: `int global = 42; static int local_static = 10;`
3. **Uninitialized data (BSS — неинициализированные данные)**
   - Глобальные и статические локальные переменные без явной инициализации.
   - ОС автоматически заполняет нулями при загрузке программы.
   - Пример: `int global; static int local_static;`
4. **Runtime heap (куча)**
   - Область для динамического выделения памяти во время выполнения.
   - Управляется аллокатором (malloc/new в C/C++, etc.).
   - Растёт вверх (в сторону больших адресов) по мере выделения памяти.
5. **Memory mapping (отображение памяти)**
   - Содержит отображённые в память файлы, прежде всего динамические библиотеки (.so/.dll).
   - Также может включать другие mmap-файлы (например, большие данные).
6. **User stack (стек пользователя)**

- Стек вызовов функций.
- Хранит локальные (авто) переменные, параметры функций, адреса возврата.
- Растёт вниз (в сторону меньших адресов).
- При завершении функции её фрейм стека очищается автоматически.

7. **Kernel virtual memory (виртуальная память ядра)**
   - Область, зарезервированная для нужд операционной системы.
   - Содержит структуры ядра, необходимые для работы процесса (например, таблицы страниц для трансляции виртуальных адресов в физические).
   - Недоступна из пользовательского режима.

Освобождение памяти

- При завершении функции: автоматически очищается её часть стека (локальные переменные).
- При завершении программы: освобождается всё виртуальное адресное пространство процесса, включая кучу и отображения.

Ссылки (references) в C++

- **Ссылки** — это псевдонимы (алиасы) уже существующих переменных.
- Ссылки **физически не существуют** как отдельные объекты: они не занимают места ни в стеке, ни в куче.
- Компилятор заменяет использование ссылки прямым обращением к оригинальной переменной.
- Основная польза:
  - Удобная и эффективная передача больших объектов в функции без копирования (передача по ссылке).
  - Избежание работы с указателями в случаях, когда нужна гарантия существования объекта.

Область видимости (Scope) переменных

- Область видимости ограничивается **блоком кода**, в котором переменная объявлена.
- Виды областей:
  - Глобальная (вне всех функций).
  - Локальная (внутри функции или блока).
  - Вложенные блоки.
- При **коллизии имён** (одинаковые имена переменных) компилятор выбирает переменную из **самой вложенной** области видимости.
- Имя переменной должно быть **уникальным** в пределах своей области видимости.
- Константы через `#define` доступны с места определения во всех модулях; отменить — `#undef`.

Инициализация

- Глобальные и статические переменные автоматически инициализируются **нулевым значением** (если не указан инициализатор).
- Локальные (автоматические) переменные **не инициализируются** по умолчанию (принцип: «не платить за неиспользуемое»).
- Чтение неинициализированной переменной — неопределённое поведение (компиляторы g++/clang++ предупреждают с `-Wall`/`-Wuninitialized`).

Время жизни и классы памяти

Класс памяти	Описание	Время жизни	Область видимости	Примечания
`auto`	Автоматическая	Временное (при входе/выходе из блока)	Локальная	По умолчанию для локальных переменных, явно не пишется
`extern`	Объявление внешней переменной	Определяется в другом месте	Глобальная (между модулями)	Не выделяет память, только объявляет
`static`	Статическая	Постоянное (на всё время выполнения программы)	Локальная — в функции; Глобальная — только в текущем файле	Инициализируется один раз

Класс памяти	Описание	Время жизни	Область видимости	Примечания
`register`	Рекомендация хранить в регистре CPU	Как `auto`	Локальная	Для оптимизации скорости (современные компиляторы часто игнорируют)

Работа с файлами в C++

Основные заголовки

- `<fstream>` — основной (включает `<ifstream>`, `<ofstream>`).
- `<ifstream>` — ввод из файла.
- `<ofstream>` — вывод в файл.

Основные операции

1. Создать объект потока (`ofstream`/`ifstream`).
2. Связать с файлом (в конструкторе или через `open()`).
3. Выполнить чтение/запись.
4. Закрыть файл (`close()`).

Запись в файл (ofstream)

```
ofstream fout("file.txt");          // создание + открытие
fout << "Текст для записи";
fout.close();
```

- По умолчанию: `ios::out | ios::trunc` (создаёт новый или перезаписывает существующий).

Чтение из файла (ifstream)

```
ifstream fin("file.txt");
string str;
fin >> str;              // по словам
getline(fin, str);       // всю строку
fin.close();
```

Проверка открытия файла

```
if (!fin.is_open()) {
    cout << "Ошибка открытия файла!";
}
```

Режимы открытия файлов (флаги из `ios_base`)

Флаг	Описание
`ios::in`	Открытие для чтения (по умолчанию для `ifstream`)
`ios::out`	Открытие для записи (по умолчанию для `ofstream`)
`ios::app`	Добавление в конец файла (запись только в конец)
`ios::ate`	Перейти в конец файла сразу после открытия (запись возможна в любом месте)
`ios::trunc`	Очистить файл при открытии (по умолчанию с `out`)
`ios::binary`	Бинарный режим

- Комбинирование: `ios::out | ios::app`.

Форматированный ввод/вывод (C-стиль)

printf() — форматированный вывод

```
printf("форматная_строка", значение1, значение2, ...);
```

Спецификаторы формата

Спецификатор	Тип данных	Пример вывода
`%d` или `%i`	int (со знаком)	392
`%u`	unsigned int	7235
`%o`	восьмеричное	657
`%x`/`%X`	шестнадцатеричное	7fa / 7FA
`%f`/`%F`	float/double	3.4563745
`%e`/`%E`	экспоненциальная форма	3.1234e+3
`%g`/`%G`	кратчайшая форма	3.12
`%c`	char	a
`%s`	строка	Hello World
`%p`	указатель	b8000000
`%%`	символ %	%

Флаги

Флаг	Эффект
`-`	Выравнивание по левому краю
`+`	Всегда показывать знак (+ или -)
(пробел)	Пробел перед положительными числами
`#`	Альтернативная форма (0, 0x и т.д.)
`0`	Заполнение нулями вместо пробелов

scanf() — форматированный ввод

```
scanf("форматная_строка", &переменная1, &переменная2, ...);
```

- Аналогичные спецификаторы, но без некоторых флагов вывода.

Работа с файлами в C (stdio.h)

Основные функции

- `FILE* fp = fopen("имя_файла", "режим");` — открытие (`r`, `w`, `a`, `r+`, `w+`, `a+`).
- `fclose(fp);` — закрытие.
- Проверка: `if (fp == NULL) ...`

Чтение

- `fscanf(fp, "%d%s", &var, str);` — форматированное.
- `fgets(str, размер, fp);` — построчно (включает `\n`).
- Возврат `EOF` при конце файла.

Запись

- `fprintf(fp, "формат", ...);` — форматированная.
- `fputs(str, fp);` — построчно (без `\n` в конце).
- `fputc(ch, fp);` — посимвольно.